

Week-1

Session-1

- 1) Write Java 8 methods using **lambda expressions** to check whether a number is **odd/even, prime/composite, or palindrome/not palindrome** based on the given operation type.

Code :

```
import java.io.*;
import java.util.*;
interface PerformOperation {
    boolean check(int a);
}
class MyMath {
    public static boolean checker(PerformOperation p, int num) {
        return p.check(num);
    }

    public PerformOperation isOdd() {
        return (a) -> a % 2 != 0;
    }

    public PerformOperation isPrime() {
        return (a) -> {
            if (a < 2)
                return false;

            for (int i = 2; i * i <= a; i++) {
                if (a % i == 0)
```

```

        return false;
    }

    return true;
};
}

public PerformOperation isPalindrome() {
    return (a) -> {
        int original = a;
        int reverse = 0;

        while (a != 0) {
            int digit = a % 10;
            reverse = reverse * 10 + digit;
            a = a / 10;
        }

        return original == reverse;
    };
}

public class Solution {

    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    }
}

```

```

int T = Integer.parseInt(br.readLine());

PerformOperation op;

boolean ret = false;

String ans = null;

while (T--> 0) {

    String s = br.readLine().trim();

    StringTokenizer st = new StringTokenizer(s);

    int ch = Integer.parseInt(st.nextToken());

    int num = Integer.parseInt(st.nextToken());

    if (ch == 1) {

        op = ob.isOdd();

        ret = ob.checker(op, num);

        ans = (ret) ? "ODD" : "EVEN";

    } else if (ch == 2) {

        op = ob.isPrime();

        ret = ob.checker(op, num);

        ans = (ret) ? "PRIME" : "COMPOSITE";

    } else if (ch == 3) {

        op = ob.isPalindrome();

        ret = ob.checker(op, num);

        ans = (ret) ? "PALINDROME" : "NOT PALINDROME";

    }

    System.out.println(ans);

}

}

}

```

Input:

5

1 4

2 5

3 898

1 3

2 12

Output:

EVEN

PRIME

PALINDROME

ODD

COMPOSITE

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

Download

```
1 5
2 1 4
3 2 5
4 3 898
5 1 3
6 2 12
```

Your Output (stdout)

```
1 EVEN
2 PRIME
3 PALINDROME
```

Session-2

- 1) Given student details such as **ID, name, and CGPA**, sort the students by **CGPA in decreasing order**. If CGPA is the same, sort by **name alphabetically**, and if the name is also the same, sort by **ID**.

Code:

```
import java.util.*;
```

```
class Student{  
    private int id;  
    private String fname;  
    private double cgpa;  
    public Student(int id, String fname, double cgpa) {  
        super();  
        this.id = id;  
        this.fname = fname;  
        this.cgpa = cgpa;  
    }  
    public int getId() {  
        return id;  
    }  
    public String getFname() {  
        return fname;  
    }  
    public double getCgpa() {  
        return cgpa;  
    }  
}
```

//Complete the code

```
public class Solution  
{
```

```
public static void main(String[] args){  
    Scanner in = new Scanner(System.in);  
    int testCases = Integer.parseInt(in.nextLine());  
  
    List<Student> studentList = new ArrayList<Student>();  
    while(testCases>0){  
        int id = in.nextInt();  
        String fname = in.next();  
        double cgpa = in.nextDouble();  
  
        Student st = new Student(id, fname, cgpa);  
        studentList.add(st);  
  
        testCases--;  
    }  
    Collections.sort(studentList, new Comparator<Student>() {  
public int compare(Student s1, Student s2) {  
    if (s1.getCgpa() != s2.getCgpa()) {  
        return Double.compare(s2.getCgpa(), s1.getCgpa());  
    }  
  
    int nameCompare = s1.getFname().compareTo(s2.getFname());  
  
    if (nameCompare != 0) {  
        return nameCompare;  
    }  
}
```

```
        return Integer.compare(s1.getId(), s2.getId());  
    }  
});
```

```
        for(Student st: studentList){  
            System.out.println(st.getFname());  
        }  
    }  
}
```

input:

5

33 Rumpa 3.68

85 Ashis 3.85

56 Samiha 3.75

19 Samara 3.75

22 Fahim 3.76

Output:

Ashis

Fahim

Samara

Samiha

Rumpa

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

```
1 5
2 33 Rumpa 3.68
3 85 Ashis 3.85
4 56 Samiha 3.75
5 19 Samara 3.75
6 22 Fahim 3.76
```

Your Output (stdout)

```
1 Ashis
2 Fahim
3 Samara
```

2) Given an integer array, rearrange it so that **all even numbers appear before all odd numbers**. Return any array that satisfies this condition.

Code:

```
class Solution {
    public int[] sortByParity(int[] nums) {

        int left = 0;
        int right = nums.length - 1;

        while (left < right) {

            if (nums[left] % 2 == 0) {
                left++;
            }
            else if (nums[right] % 2 != 0) {
                right--;
            }
        }
    }
}
```

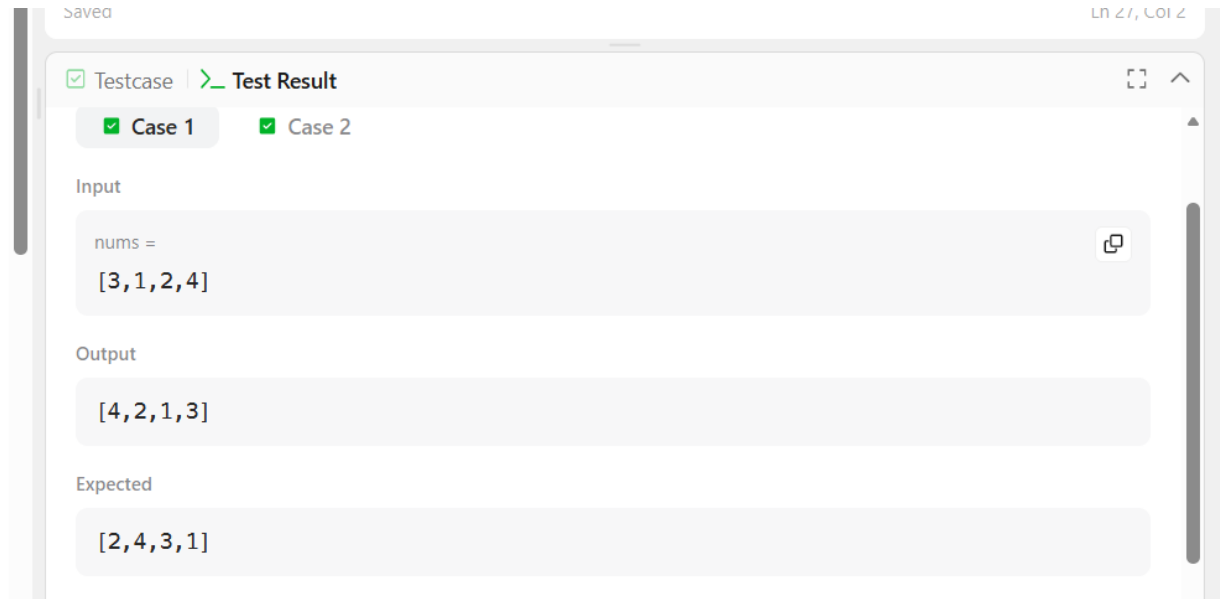
```
    }  
    else {  
        int temp = nums[left];  
        nums[left] = nums[right];  
        nums[right] = temp;  
  
        left++;  
        right--;  
    }  
}  
  
return nums;  
}
```

Input:

nums = [3,1,2,4]

Output:

[2,4,3,1]



Session-3

1) Problem Statement

Create a **Comparator** to sort Player objects by **score in decreasing order**. If two players have the same score, sort them by **name in alphabetical order**.

Code:

```
import java.util.*;
```

```
class Player {  
    String name;  
    int score;  
  
    Player(String name, int score) {  
        this.name = name;  
        this.score = score;  
    }  
}
```

```
class Checker implements Comparator<Player> {
```

```

public int compare(Player a, Player b) {

    if (a.score != b.score) {
        return Integer.compare(b.score, a.score);
    }
    return a.name.compareTo(b.name);
}

}

public class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();

        Arrays.sort(player, checker);
    }
}

```

```
for(int i = 0; i < player.length; i++){  
    System.out.printf("%s %s\n", player[i].name, player[i].score);  
}  
}
```

Input:

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Output:

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

1 5

2 amy 100

3 david 100

4 heraldo 50

5 aakansha 75

6 aleksa 150

Download

Your Output (stdout)

1 aleksa 150

2 amy 100

3 david 100

Session-5

1) Problem Statement

Given a 2D array representing customers' money in different bank accounts, calculate each customer's total wealth. Return the **maximum wealth** among all customers.

Code:

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        int max = 0;  
  
        for (int i = 0; i < accounts.length; i++) {  
            int sum = 0;  
  
            for (int j = 0; j < accounts[i].length; j++) {  
                sum += accounts[i][j];  
            }  
        }  
    }  
}
```

```
        max = Math.max(max, sum);
    }

    return max;
}
```

Input:

```
accounts = [[1,5],[7,3],[3,5]]
```

Output:

10

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

accounts =
[[1,2,3] , [3,2,1]]

Output

6

Week-2

Session-6

1) Problem Statement

Given a zero-based permutation `nums`, create a new array `ans` where each element is `ans[i] = nums[nums[i]]`. Return the resulting array.

Code:

```
class Solution {  
    public int[] buildArray(int[] nums) {  
        int[] ans = new int[nums.length];  
  
        for (int i = 0; i < nums.length; i++) {  
            ans[i] = nums[nums[i]];  
        }  
  
        return ans;  
    }  
}
```

Input:

`nums = [0,2,1,5,3,4]`

Output:

`[0,1,2,4,5,3]`

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =
[0,2,1,5,3,4]

Output

[0,1,2,4,5,3]

Expected

[0,1,2,4,5,3]

2) Given an array of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$, rearrange it into $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Code:

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] ans = new int[nums.length];  
        int j = 0;  
  
        for (int i = 0; i < n; i++) {  
            ans[j++] = nums[i];  
            ans[j++] = nums[i + n];  
        }  
  
        return ans;  
    }  
}
```

Input:

nums = [2,5,1,3,4,7], n = 3

Output:

[2,3,5,4,1,7]



Session-7

1) Problem Statement

Given a sorted integer array, remove duplicate elements **in-place** so that each unique element appears only once. Return the number of unique elements k, with the first k positions containing the unique values.

Code:

```
class Solution {  
  
    public int removeDuplicates(int[] nums) {  
  
        int k = 1;  
  
        for (int i = 1; i < nums.length; i++) {  
            if (nums[i] != nums[i - 1]) {  
                nums[k] = nums[i];  
                k++;  
            }  
        }  
    }  
}
```

```
        return k;
    }
}
```

Input:

nums = [0,0,1,1,1,2,2,3,3,4]

Output:

5

nums = [0,1,2,3,4]



Session-8

1) Problem Statement

Given two players with a **name and score**, compare them for sorting. Sort players by **score in descending order**, and if scores are equal, sort by **name in alphabetical order**.

Code:

```
class Solution {
    public int largestAltitude(int[] gain) {
        int altitude = 0;
        int maxAltitude = 0;
```

```

    for (int i = 0; i < gain.length; i++) {
        altitude += gain[i];
        maxAltitude = Math.max(maxAltitude, altitude);
    }

    return maxAltitude;
}
}

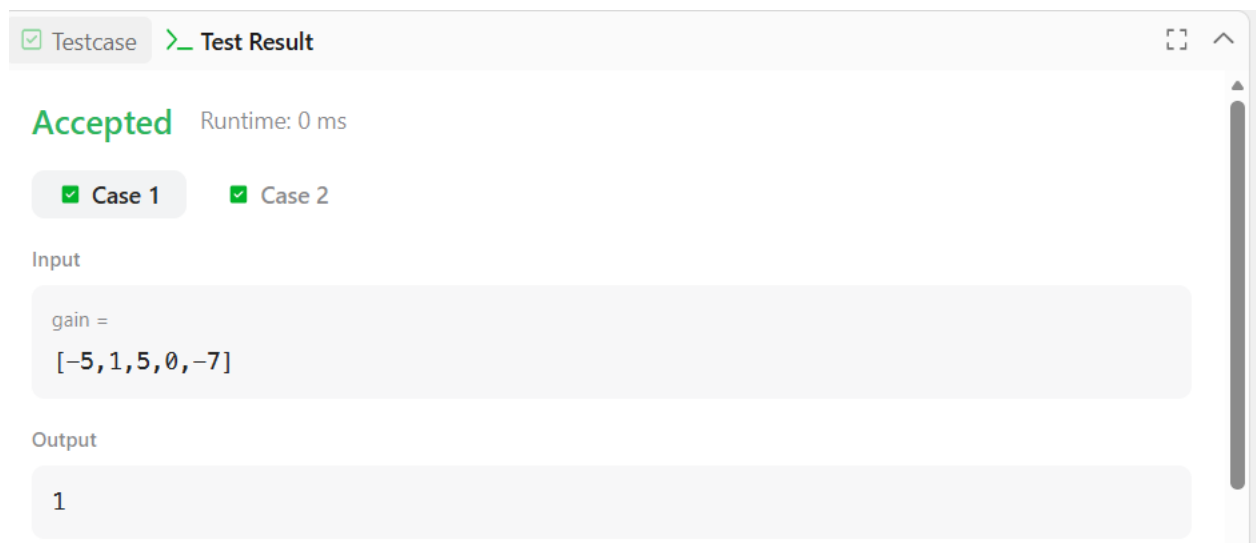
```

Input:

gain = [-5,1,5,0,-7]

Output:

1



The screenshot shows a web-based code execution environment. At the top, there are two tabs: 'Testcase' (selected) and 'Test Result'. Below the tabs, the word 'Accepted' is displayed in green, followed by 'Runtime: 0 ms'. Underneath, there are two buttons: 'Case 1' and 'Case 2', both with green checkmarks. The 'Input' section shows 'gain =' followed by the array '[-5,1,5,0,-7]'. The 'Output' section shows the number '1'. The interface includes a scrollbar on the right side.

Session-9

1) Problem Statement

Given an integer array `nums` and an integer `k`, find and return the **k elements that occur most frequently** in the array. The answer can be returned in any order.

Code:

```
import java.util.*;
```

```
class Solution {
```

```
    public int[] topKFrequent(int[] nums, int k) {
```

```
        HashMap<Integer, Integer> map = new HashMap<>();
```

```
        for (int num : nums) {
```

```
            map.put(num, map.getOrDefault(num, 0) + 1);
```

```
        }
```

```
        PriorityQueue<Integer> pq = new PriorityQueue<>(  
            (a, b) -> map.get(a) - map.get(b)
```

```
        );
```

```
        for (int num : map.keySet()) {
```

```
            pq.add(num);
```

```
            if (pq.size() > k) {
```

```
                pq.poll();
```

```
            }
```

```
        }
```

```
        int[] result = new int[k];
```

```
        for (int i = k - 1; i >= 0; i--) {
```

```
            result[i] = pq.poll();
```

```
        }
```

```
        return result;
    }
}
```

Input:

nums = [1,1,1,2,2,3]

k = 2

Output:

☑ Testcase | >_ Test Result

Accepted Runtime: 1 ms

☑ Case 1

☑ Case 2

☑ Case 3

Input

```
nums =  
[1,1,1,2,2,3]
```

```
k =  
2
```

Output

```
[1,2]
```

Expected

```
[1,2]
```

Session-10

1) Problem Statement

Given an array of n integers and a subarray size m , find the **maximum number of unique integers** present in any contiguous subarray of size m . Use a **Deque** to efficiently maintain the sliding window.

Code:

```
import java.util.*;
```

```
public class test {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
  
        Deque<Integer> deque = new ArrayDeque<>();  
        HashSet<Integer> set = new HashSet<>();  
  
        int n = in.nextInt();  
        int m = in.nextInt();  
  
        int max = 0;  
  
        for (int i = 0; i < n; i++) {  
            int num = in.nextInt();  
  
            deque.addLast(num);  
            set.add(num);  
  
            if (deque.size() > m) {  
                int removed = deque.removeFirst();  
  
                if (!deque.contains(removed)) {  
                    set.remove(removed);  
                }  
            }  
  
            if (deque.size() == m) {
```

```

        max = Math.max(max, set.size());
    }
}

System.out.println(max);
}
}

```

Input:

6 3

5 3 5 2 3 2

Output:

3

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

```

1 6 3
2 5 3 5 2 3 2

```

Your Output (stdout)

```

1 3

```

Expected Output

[Download](#)

```

1 3

```

2) Problem Statement

Given multiple pairs of strings, use a **HashSet** to store only unique pairs. After reading each pair, print the current number of unique pairs.

Code:

```
import java.io.*;
```

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int t = s.nextInt();

        HashSet<String> set = new HashSet<>();

        for (int i = 0; i < t; i++) {
            String left = s.next();
            String right = s.next();

            set.add(left + " " + right);

            System.out.println(set.size());
        }
    }
}
```

Input:

5

john tom

john mary

john tom

mary anna

mary anna

Output:

1
2
2
3
3

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

```
1 5
2 john tom
3 john mary
4 john tom
5 mary anna
6 mary anna
```

Your Output (stdout)

```
1 1
2 2
3 2
4 3
5 3
```

Week-3

Session-11

1) Problem Statement

Given a date consisting of **month, day, and year**, determine and return the **day of the week** for that date in uppercase.

Code:

```
import java.io.*;
import java.util.*;
```

```
class Result {
```

```

public static String findDay(int month, int day, int year) {
    Calendar calendar = Calendar.getInstance();
    calendar.set(year, month - 1, day);

    String[] days = {
        "SUNDAY",
        "MONDAY",
        "TUESDAY",
        "WEDNESDAY",
        "THURSDAY",
        "FRIDAY",
        "SATURDAY"
    };

    return days[calendar.get(Calendar.DAY_OF_WEEK) - 1];
}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

```

```
int month = Integer.parseInt(firstMultipleInput[0]);
int day = Integer.parseInt(firstMultipleInput[1]);
int year = Integer.parseInt(firstMultipleInput[2]);

String res = Result.findDay(month, day, year);

bufferedWriter.write(res);
bufferedWriter.newLine();

bufferedReader.close();
bufferedWriter.close();
}
}
```

Input:

08 05 2015

Output:

WEDNESDAY

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

[Download](#)

1 08 05 2015

Your Output (stdout)

1 WEDNESDAY

Expected Output

[Download](#)

1 WEDNESDAY

2) Problem Statement

Given two valid dates in YYYY-MM-DD format, calculate and return the **absolute number of days between the two dates**.

Code:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

class Solution {
    public int daysBetweenDates(String date1, String date2) {
        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);

        return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
    }
}
```

Input:

date1 = "2019-06-29"

date2 = "2019-06-30"

Output:

1



Session-12

1) Problem Statement

Given a date in YYYY-MM-DD format, return the **day number of that year**. For example, January 9 is day 9, while February 10 is day 41.

Code:

```
class Solution {  
    public int dayOfYear(String date) {  
        int year = Integer.parseInt(date.substring(0, 4));  
        int month = Integer.parseInt(date.substring(5, 7));  
        int day = Integer.parseInt(date.substring(8, 10));  
  
        int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};  
  
        if (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)) {  
            days[1] = 29;  
        }  
    }  
}
```

```

    int result = day;

    for (int i = 0; i < month - 1; i++) {
        result += days[i];
    }

    return result;
}
}

```

Input:

date = "2019-02-10"

Output:

41

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

date =
"2019-01-09"

Output

9

Expected

9

2) Problem Statement

Given a valid date as **day, month, and year**, determine the corresponding **day of the week**. Return the weekday name such as Sunday, Monday, etc.

Code:

```
import java.time.LocalDate;
```

```
class Solution {  
    public String dayOfTheWeek(int day, int month, int year) {  
        return LocalDate.of(year, month, day)  
            .getDayOfWeek()  
            .toString()  
            .charAt(0) +  
            LocalDate.of(year, month, day)  
            .getDayOfWeek()  
            .toString()  
            .substring(1).toLowerCase();  
    }  
}
```

Input:

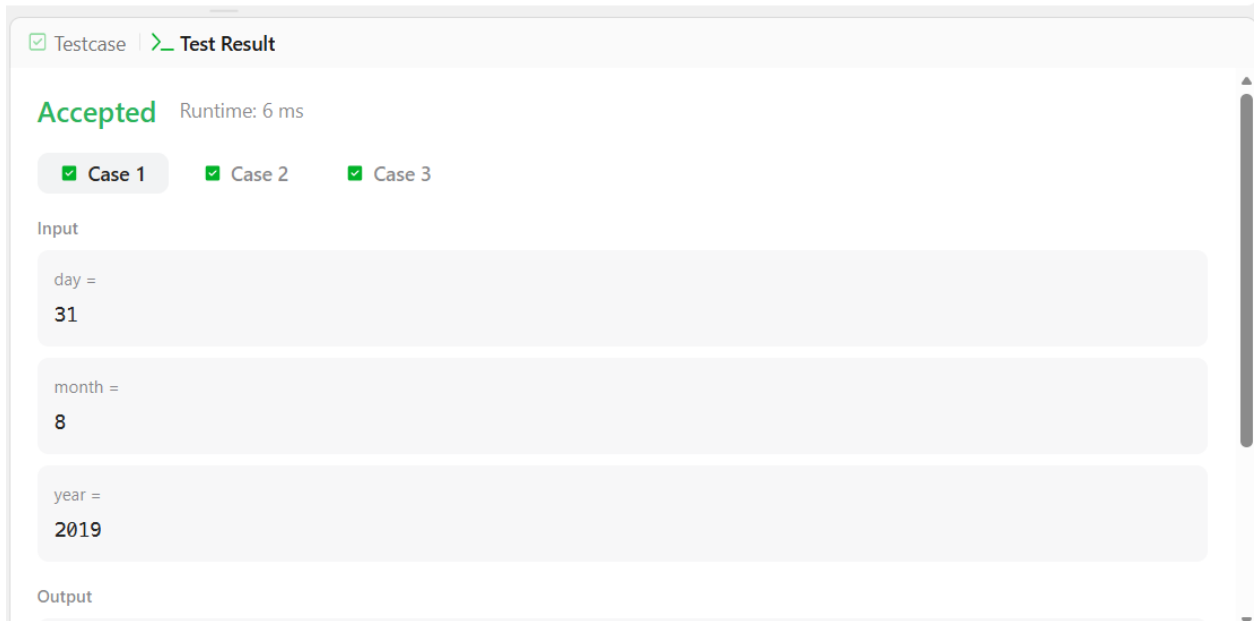
day = 31

month = 8

year = 2019

Output:

"Saturday"



Session-13

1) Problem Statement

Implement a **Priority Queue** to manage students based on their priority. Students are served by **highest CGPA**, then **alphabetical name**, and finally **lowest ID** when both are equal.

Code:

```
import java.io.*;
import java.util.*;
```

```
class Student {
    int id;
    String name;
    double cgpa;

    Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
}
```

```
}  
}
```

```
public class Solution {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();  
  
        PriorityQueue<Student> pq = new PriorityQueue<>((a, b) -> {  
            if (a.cgpa != b.cgpa) {  
                return Double.compare(b.cgpa, a.cgpa);  
            }  
  
            int nameCompare = a.name.compareTo(b.name);  
  
            if (nameCompare != 0) {  
                return nameCompare;  
            }  
  
            return Integer.compare(a.id, b.id);  
        });  
  
        for (int i = 0; i < n; i++) {  
            String event = sc.next();  
  
            if (event.equals("ENTER")) {
```

```

        String name = sc.next();

        double cgpa = sc.nextDouble();

        int id = sc.nextInt();


        pq.add(new Student(id, name, cgpa));
    } else {
        if (!pq.isEmpty()) {
            pq.poll();
        }
    }
}

if (pq.isEmpty()) {
    System.out.println("EMPTY");
} else {
    while (!pq.isEmpty()) {
        System.out.println(pq.poll().name);
    }
}

sc.close();
}
}

```

Input:

12

ENTER John 3.75 50

ENTER Mark 3.8 24

ENTER Shafaet 3.7 35

SERVED

SERVED

ENTER Samiha 3.85 36

SERVED

ENTER Ashley 3.9 42

ENTER Maria 3.6 46

ENTER Anik 3.95 49

ENTER Dan 3.95 50

SERVED

Output:

Dan

Ashley

Shafaet

Maria

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

```
1 Dan
2 Ashley
3 Shafaet
4 Maria
```

Expected Output

```
1 Dan
2 Ashley
3 Shafaet
4 Maria
```

[Download](#)

2) Problem Statement

Given multiple lines of integers stored using **ArrayList**, answer queries asking for the value at a specific position in a specific line. If the requested position does not exist, print **ERROR!**.

Code:

```
import java.io.*;

import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<ArrayList<Integer>> list = new ArrayList<>();

        for (int i = 0; i < n; i++) {

            int d = sc.nextInt();

            ArrayList<Integer> row = new ArrayList<>();

            for (int j = 0; j < d; j++) {

                row.add(sc.nextInt());

            }

            list.add(row);

        }

        int q = sc.nextInt();
```

```

for (int i = 0; i < q; i++) {
    int x = sc.nextInt();
    int y = sc.nextInt();

    if (x >= 1 && x <= list.size() && y >= 1 && y <= list.get(x - 1).size()) {
        System.out.println(list.get(x - 1).get(y - 1));
    } else {
        System.out.println("ERROR!");
    }
}

sc.close();
}
}

```

Input:

5

5 41 77 74 22 44

1 12

4 37 34 36 52

0

3 20 22 33

5

1 3

3 4

3 1

4 3

5 5

Output:

74

52

37

ERROR!

ERROR!

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Your Output (stdout)

1	74
2	52
3	37
4	ERROR!
5	ERROR!

Expected Output

1	74
2	52
3	37
4	ERROR!
5	ERROR!

Download

Session-14

1) Problem Statement

Given a list of non-negative integers, rearrange them to form the **largest possible number**. Return the result as a **string** because the number can be very large.

Code:

```
import java.util.*;
```

```

class Solution {
    public String largestNumber(int[] nums) {
        String[] arr = new String[nums.length];

        for (int i = 0; i < nums.length; i++) {
            arr[i] = String.valueOf(nums[i]);
        }

        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));

        if (arr[0].equals("0")) {
            return "0";
        }

        StringBuilder result = new StringBuilder();

        for (String s : arr) {
            result.append(s);
        }

        return result.toString();
    }
}

```

Input:

nums = [3,30,34,5,9]

Output:

"9534330"

☒ Testcase | ☒ Test Result

Accepted Runtime: 1 ms

☒ Case 1 ☒ Case 2

Input

nums =
[10,2]

Output

"210"

Expected

"210"

2) Problem Statement

Create a **Comparator** to sort players by **score in decreasing order**. If two players have the same score, sort them by their **name in alphabetical order**.

Code:

```
import java.util.*;
```

```
class Player {
```

```
    String name;
```

```
    int score;
```

```
    Player(String name, int score) {
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
class Checker implements Comparator<Player> {  
    public int compare(Player a, Player b) {  
        if (a.score != b.score) {  
            return Integer.compare(b.score, a.score);  
        }  
        return a.name.compareTo(b.name);  
    }  
}
```

```
public class Solution {  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
  
        int n = scan.nextInt();  
        Player[] player = new Player[n];  
  
        for (int i = 0; i < n; i++) {  
            player[i] = new Player(scan.next(), scan.nextInt());  
        }  
  
        Arrays.sort(player, new Checker());  
  
        for (Player p : player) {  
            System.out.println(p.name + " " + p.score);  
        }  
}
```

```
        scan.close();  
    }  
}
```

Input:

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Output:

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

 **Sample Test case 0**

Input (stdin)[Download](#)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

Your Output (stdout)

```
1 aleksa 150
2 amy 100
3 david 100
```

Session-15

2) Problem Statement

Given arrays of names and distinct heights, sort and return the **names in descending order of height**. The tallest person should appear first.

Code:

```
import java.util.*;
```

```
class Solution {
```

```
    public String[] sortPeople(String[] names, int[] heights) {
```

```
        Integer[] index = new Integer[names.length];
```

```
        for (int i = 0; i < names.length; i++) {
```

```
            index[i] = i;
```

```
        }
```

```
Arrays.sort(index, (a, b) -> heights[b] - heights[a]);
```

```
String[] result = new String[names.length];
```

```
for (int i = 0; i < names.length; i++) {  
    result[i] = names[index[i]];  
}
```

```
return result;
```

```
}
```

```
}
```

Input:

names = ["Mary","John","Emma"]

heights = [180,165,170]

Output:

["Mary","Emma","John"]

Testcase

Test Result

Case 1

Case 2

Input

names =
["Mary","John","Emma"]

heights =
[180,165,170]

Output

["Mary","Emma","John"]

Expected

["Mary","Emma","John"]

